

```

# -----
# Consumer Choice
# -----

import os
import sys

import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize, LinearConstraint, Bounds

# Printing the outputs as a text file if ==1
output_file = 0

# -----
# Problem Set-up
# -----
alpha = 0.7

def fun(x):
    return -(x[0] ** alpha) * (x[1] ** (1 - alpha)) # negative

A = np.array([1.0, 1.0]) # A = [p1, p2]
b = 1.0                 # b = [m]

x0 = np.array([0.1, 0.1])

# budget constraint: p1*x1 + p2*x2 <= m
budget = LinearConstraint([A], -np.inf, b)

# non-negativity: x1 >= 0, x2 >= 0
bounds = Bounds([0.0, 0.0], [np.inf, np.inf])

# -----
# minimize (trust-constr)
# -----
result = minimize(fun, x0, method='trust-constr',
                  constraints=[budget], bounds=bounds)

x = result.x
fval = result.fun
lam_budget = result.v[0] # multiplier for the budget constraint
lam_bounds = result.v[1] # multipliers for [x1>=0, x2>=0]

if output_file == 1:
    dfile = "consumer_choice_python.txt"
    if os.path.exists(dfile):
        os.remove(dfile)
    sys.stdout = open(dfile, "w")

# -----
# Output
# -----
print(f"alpha={alpha:.3f}, p1={A[0]:.3f}, p2={A[1]:.3f}, m={b:.3f}")
print(f"Optimal consumption: {x[0]:.3f},{x[1]:.3f}")
print(f"Utility: {-fval:.5f}")
print(f"Multiplier (budget const): {lam_budget[0]:.5f}")

```

```

print(f"Multiplier (non-negativity const): {lam_bounds[0]:.5f}, {lam_bounds[1]:.5f}")

if output_file == 1:
    sys.stdout.close()
    sys.stdout = sys.__stdout__

# -----
# Plot
# -----
x1_vec = np.linspace(0, b / A[0], 100)
x2_vec = np.linspace(0, b / A[1], 100)
x1_mesh, x2_mesh = np.meshgrid(x1_vec, x2_vec)
utility_mesh = (x1_mesh ** alpha) * (x2_mesh ** (1 - alpha))

fig, ax = plt.subplots()
ax.grid(True)
ax.plot(x1_vec, (b - A[0] * x1_vec) / A[1], 'k', linewidth=1) # budget constraint
ax.contour(x1_mesh, x2_mesh, utility_mesh, levels=[-fval],
           colors='b', linewidths=1) # indifference curve
ax.contour(x1_mesh, x2_mesh, utility_mesh,
           levels=[0.75 * (-fval), 1.25 * (-fval)],
           colors='k', linewidths=1, linestyles='--') # neighboring indifference curves
ax.plot(x[0], x[1], 'ro', linewidth=1) # optimal point
ax.set_xlabel(r'$x_{1}$', fontsize=12)
ax.set_ylabel(r'$x_{2}$', fontsize=12)
ax.set_title('Consumer Choice', fontsize=14)
ax.set_xlim([0, b / A[0]])
ax.set_ylim([0, b / A[1]])
plt.show()

```